

Range Minimum Queries

Feyzi Eren Gündoğdu

Karlsruhe Institute of Technology, Germany

1 Introduction

This report presents a comparison of different data structures and their performance for Range Minimum Queries (RMQ) in C++.

Six different data structures, including two naive and four advanced approaches, are discussed in this report. The naive algorithms, *OnTheFlyNaive* and *PrecomputedNaive*, are included to provide upper-bound performance comparisons for RMQ. The advanced data structures considered are *SparseTable*, *SegmentTree*, *BlockSparseTable*, and *CartesianTree*.

For some of the advanced data structures, different variants were evaluated and are represented by different suffixes (e.g., *BST16* denotes a Block Sparse Table with a block size of 16). n represents the input size, B represents block size.

Algorithm	Space overhead (bits)	Query time
OnTheFlyNaive	$O(1)$	$O(n)$
PrecomputedNaive	$O(n^2 \log n)$	$O(1)$
Sparse Table	$O(n \log^2 n)$	$O(1)$
Segment Tree	$O(n \log n)$	$O(\log n)$
Blocks	$O\left(\frac{n}{s} \log^2 n\right)$	$O(s)$
Cartesian Trees	$O(n \log n)$	$O(1)$

■ **Table 1** Theoretical bounds.

TODO FILES

2 Methods

In this section, each data structure is discussed in detail. Only the non-naive aspects of the implementation and analysis are presented.

2.1 OnTheFlyNaive

This algorithm stores only a pointer to the input data and answers each query by scanning the requested range on the fly. Since no preprocessing is performed and no additional RMQ data structure is stored, it requires only $O(1)$ extra space. In the worst case, a query may scan the whole array, so the query time is $O(n)$.

2.2 PrecomputedNaive

This algorithm stores the answers for all possible RMQ instances in a triangular table. For every left endpoint l , it stores the minimum index of each range $[l, r]$ with $l \leq r$. Since only ranges with $l \leq r$ are valid, it is sufficient to store the upper triangular part of the full $n \times n$ table. In the implementation, the entry for the range $[l, r]$ is stored at position

`table[l][r - l]`.

Equivalently, in a flat triangular layout, the range $[l, r]$ can be mapped to a single array position by

$$l \cdot n - \frac{l(l-1)}{2} + (r-l).$$

A query can then be answered by directly accessing the corresponding table entry. The algorithm theoretically requires $O(n^2 \log n)$ bits of space and query time is $O(1)$. For this implementation, each entry is stored as a fixed-width `uint32_t` instead of using exactly $\lceil \log_2 n \rceil$ bits per entry. Therefore, the expected space usage follows

$$O\left(\frac{n(n+1)}{2}\right)$$

machine words, while the corresponding theoretical space complexity is

$$O\left(\frac{n(n+1)}{2} \log n\right) = O(n^2 \log n).$$

3 Results

3.1 OnTheFlyNaive

3.1.1 Space

In the bits-per-element plot in Figure 1, the space usage appears to converge to 0. This is expected, because the method uses only constant additional space. In this implementation, the only stored data is a pointer to the input array, which takes approximately 8 bytes. Therefore, the plotted value is approximately $64/n$ bits per element, which tends to 0 as n grows. The absolute space overhead is shown in Figure 2. There, the space usage remains constant at approximately 64 bits, matching the theoretical $O(1)$ extra space bound.

3.1.2 Time

The query time in Figure 3 grows with n , which is consistent with the $O(n)$ worst-case query time. This is because each query is answered by scanning the queried range directly.

3.2 PrecomputedNaive

3.2.1 Space

The results from `data.csv` indicate that the measured space usage matches the expected implementation-level bound. For example, for $n = 10000$, the table contains

$$\frac{10000 \cdot 10001}{2}$$

entries. Since each entry is stored as a `uint32_t`, each entry occupies 4 bytes. Therefore, the expected space usage is approximately

$$4 \cdot \frac{10000 \cdot 10001}{2} = 200020000$$

bytes, plus a small constant overhead. This matches the measured value 200020040 bytes. Although the curve appears almost linear in the log-log Figure 2, this does not indicate linear space growth in n . The measured space usage of PrecomputedNaive is proportional to

$$32 \cdot \frac{n(n+1)}{2}$$

bits, which is $\Theta(n^2)$. On a log-log plot, polynomial growth appears as a straight line; in this case, the slope corresponds to quadratic growth.

3.2.2 Time

For the query time, the theoretical $O(1)$ bound is only approximately visible in the measurements. Each query performs a constant number of table lookups, but the measured running time is affected by cache and memory hierarchy effects as the table grows. Nevertheless, Figure 3 shows that PrecomputedNaive behaves near-constantly. The query time of PrecomputedNaive increases sharply from $n = 1000$ to $n = 3000$, but grows more slowly afterwards. This can be explained by cache effects. For small inputs, a large part of the precomputed table can still be served from cache. Once the table exceeds the relevant cache levels, random table lookups become much more expensive. After this point, the queries are already mostly memory-latency bound, so increasing n further does not necessarily lead to a proportional increase in query time. Thus, the observed behavior is consistent with an $O(1)$ query algorithm affected by the memory hierarchy.

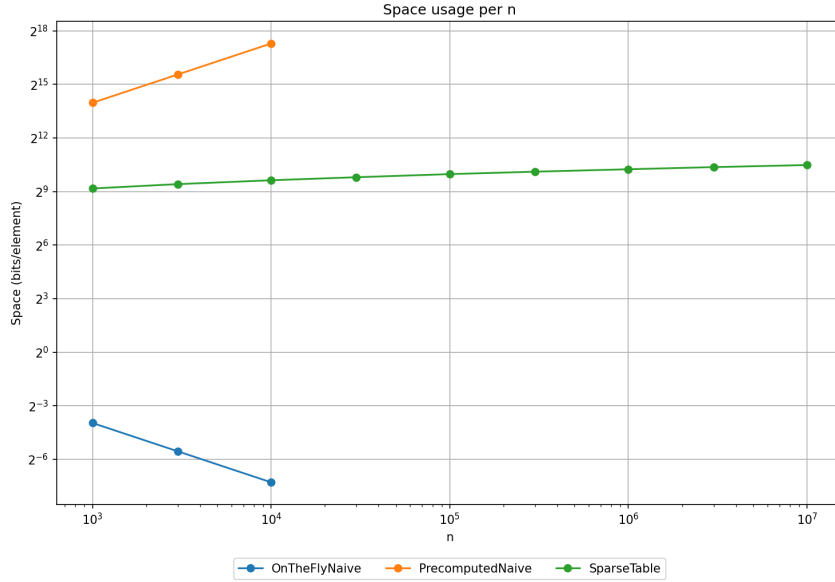


Figure 1 space usage

4 Range Minimum Queries

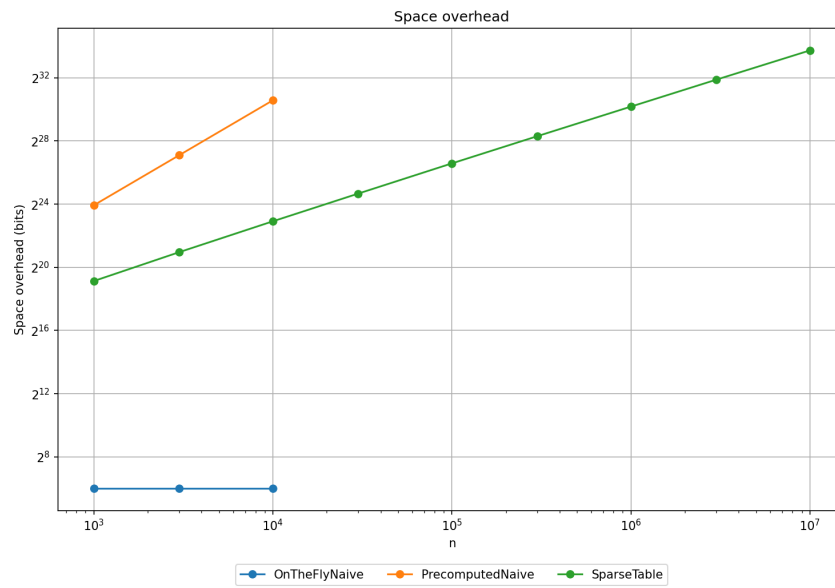


Figure 2 space overhead

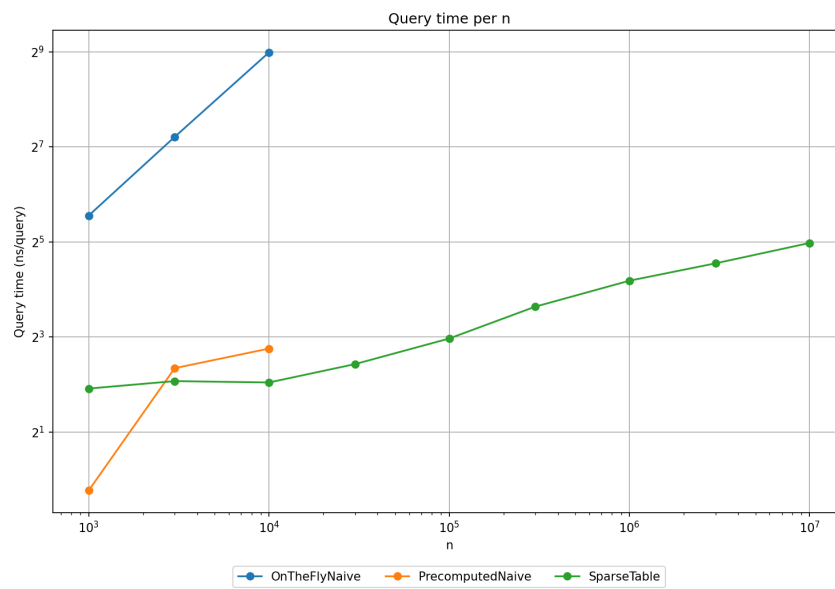


Figure 3 query time

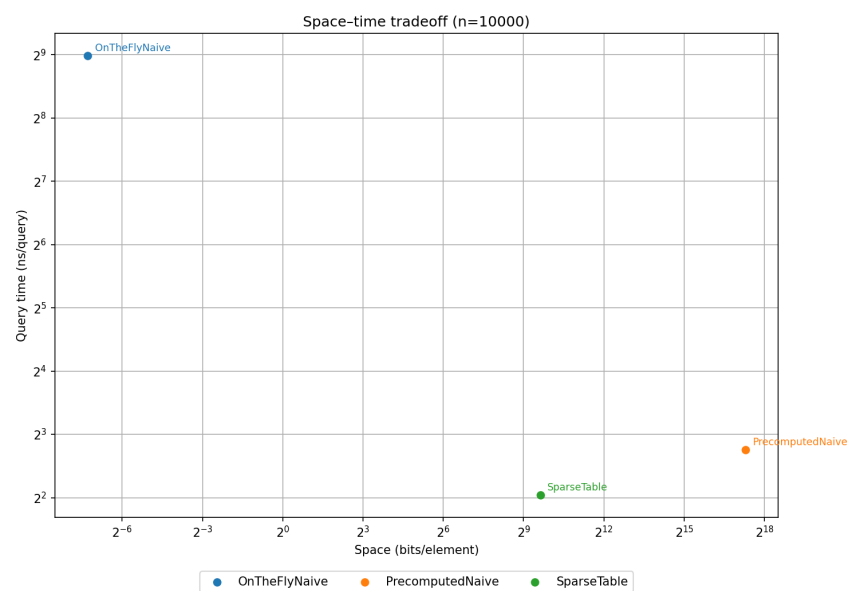


Figure 4 space-time trade-off